Scope & Hoisting

Where names live, and what runs before your code

Global `const g`

Function `var f`

Block `let b`

Name lookup goes outward

1. block
2. function
3. global
4. ✕ not found: ReferenceError

Declaration	Before its line	Early access
`function f() {}`	whole function	✓ works
`var f = () => {}`	undefined	✕ TypeError
`let · const · class`	TDZ	✕ ReferenceError

1 Shadowing

```
let x = 1;
{ let x = 2; x; }   // 2
x;                  // 1
```

2 TDZ: time, not position

```
function show() { return k; }
const k = 5;
show();   // 5
```

TRAP What does this log?

```
console.log(typeof f, typeof g);
function f() {}
var g = () => {};
```

`function` `undefined`

f is hoisted whole, g only as undefined.

KEY TAKEAWAY

- `function` Callable early
- `var` Reads as undefined
- `let/const` Throws early (TDZ)

? Follow-up questions to expect

- **What is lexical scope?** → Scope is fixed by where the code is written.
- **Does hoisting move code to the top?** → No. Declarations are registered first.
- **What is the TDZ?** → Time before the line runs. Access throws, even typeof.
- **A function inside a block?** → Sloppy: leaks as var. Strict: block only.